Universidad de San Carlos de Guatemala

Facultad de Ingeniería

Ingeniería en Ciencias y Sistemas

Lenguajes Formales y de Programación, Sección A

# Manual Técnico

# Práctica 1: Sistema de Análisis de Rendimiento Académico

Angel Sebastian Mazariegos Pérez

202300583

2/03/2026

Structs usados:

```
// estructura para guardar los datos de un estudiante
struct Estudiante {
    int carnet;
    string nombre;
    string apellido;
    string carrera;
    int semestre;
};

// estructura para guardar los datos de un curso
struct Curso {
    int codigo;
    string nombre;
    int creditos;
    int semestre;
    string carrera;
};

// estructura para guardar las notas de un estudiante en un curso
struct Nota {
    int carnet;
    int codigo_curso;
    double nota;
    string ciclo;
    int año;
};
```

Vectores usados:

```
// vectores globales para almacenar todos los datos
vector<Estudiante> estudiantes;
vector<Curso> cursos;
vector<Nota> notas;
```

FUNCIONES PRINCIPALES DEL PROGRAMA

1. split(const string& texto, char delimitador) → vector<string>

Descripción: Función auxiliar que divide un string en múltiples partes usando un delimitador específico (similar al split de Python). Es esencial para parsear los archivos CSV (.lfp).

```cpp
// función auxiliar para separar un string por un delimitador (como el split de
vector<string> split(const string& texto, char delimitador) {
    // acá voy guardando cada pedacito que salga del texto
    vector<string> resultado;

    // convierto el texto a stream para leerlo por partes con getline
    stringstream ss(texto);

    // esta variable temporal me guarda cada parte antes de meterla al vector
    string item;

    // mientras getline encuentre otro pedazo separado por el delimitador, sigo
    // ejemplo rápido: "a,b,c" con ',' -> primero "a", luego "b", luego "c"
    while (getline(ss, item, delimitador)) {
        // aquí solo lo meto al vector final
        resultado.push_back(item);
    }

    // al final regreso todo ya separado
```

2. calcularPromedio(const vector<double>& valores) → double

Descripción: Calcula el promedio de un conjunto de valores. Retorna 0 si el vector está vacío.

```cpp
// función para sacar el promedio de una lista de notas
double calcularPromedio(const vector<double>& valores) {
    // si no hay valores, regreso 0 para evitar división entre 0
    if (valores.empty()) return 0.0;

    // voy sumando todo para luego dividir entre la cantidad
    double suma = 0.0;
    for (double valor : valores) {
        suma += valor;
    }

    return suma / valores.size();
}
```

3. calcularMaximo(const vector<double>& valores) → double

Descripción: Encuentra el valor máximo en un vector usando max_element de STL.

```
// función para sacar la nota máxima de una lista
double calcularMaximo(const vector<double>& valores) {
    // si está vacío, no hay máximo real, entonces regreso 0
    if (valores.empty()) return 0.0;

    return *max_element(valores.begin(), valores.end());
}
```

4. calcularMinimo(const vector<double>& valores) → double

Descripción: Encuentra el valor mínimo en un vector usando min_element de STL.

```
// función para sacar la nota mínima de una lista
double calcularMinimo(const vector<double>& valores) {
    // si está vacío, no hay mínimo real, entonces regreso 0
    if (valores.empty()) return 0.0;

    return *min_element(valores.begin(), valores.end());
}
```

5. calcularMediana(vector<double> valores) → double

Descripción: Calcula la mediana de un conjunto de valores. Ordena el vector y retorna el valor central (o promedio de dos centrales si es par). Si cantidad es impar: retorna elemento del medio. Si cantidad es par: retorna promedio de dos elementos centrales.

```
// función para sacar la mediana de una lista
double calcularMediana(vector<double> valores) {
    // si no hay datos, regreso 0
    if (valores.empty()) return 0.0;

    // ordeno para poder encontrar el centro
    sort(valores.begin(), valores.end());

    int n = static_cast<int>(valores.size());

    // si la cantidad es impar, la mediana es el dato del centro
    if (n % 2 != 0) {
        return valores[n / 2];
    }

    // si es par, saco el promedio de los dos del centro
    return (valores[(n / 2) - 1] + valores[n / 2]) / 2.0;
}
```

6. calcularDesviacionEstandar(const vector<double>& valores) → double

Descripción: Calcula la desviación estándar (raíz cuadrada de la varianza). Mide qué tan dispersos están los datos respecto al promedio.

```
// función para sacar la desviación estándar (poblacional)
double calcularDesviacionEstandar(const vector<double>& valores) {
    // con lista vacía no hay desviación
    if (valores.empty()) return 0.0;

    double promedio = calcularPromedio(valores);
    double sumaDiferencias = 0.0;

    // aquí voy acumulando (xi - promedio)^2
    for (double valor : valores) {
        double diferencia = valor - promedio;
        sumaDiferencias += (diferencia * diferencia);
    }

    // varianza poblacional = suma / n, luego raíz para desviación
    double varianza = sumaDiferencias / valores.size();
    return sqrt(varianza);
}

```

7. obtenerCreditosCurso(int codigoCurso) → int

Descripción: Función de búsqueda que encuentra un curso por código y retorna su cantidad de créditos. Retorna 0 si no encuentra el curso.

```
// función auxiliar para buscar créditos de un curso por código
int obtenerCreditosCurso(int codigoCurso) {
    // recorro la lista de cursos hasta encontrar el código
    for (const Curso& curso : cursos) {
        if (curso.codigo == codigoCurso) {
            return curso.creditos;
        }
    }

    // si no lo encontré, regreso 0 para no romper cálculos
    return 0;
}
```

8. cargarEstudiantes(const string& ruta) → void

Descripción: Lee el archivo de estudiantes (.lfp) desde la ruta especificada y carga todos los registros en el vector global estudiantes.

Formato esperado: carnet,nombre,apellido,carrera,semestre

Validación: Verifica que el archivo exista; muestra mensaje de error si no.

```
662
663  // función para cargar el archivo de estudiantes
664  void cargarEstudiantes(const string& ruta) {
665      ifstream archivo(ruta);
666
667      // verificamos si el archivo se abrió correctamente
668      if (!archivo.is_open()) {
669          cout << "error: no se pudo abrir el archivo de estudiantes" << endl;
670          return;
671      }
672
673      string linea;
674      int contador = 0;
675
676      // leemos línea por línea del archivo
677      while (getline(archivo, linea)) {
678          // saltamos líneas vacías
679          if (linea.empty()) continue;
680
681          // separamos la línea por comas
682          vector<string> datos = split(linea, ',');
683
684          // verificamos que tengamos todos los campos
685          if (datos.size() != 5) {
686              cout << "advertencia: línea con formato incorrecto: " << linea << endl;
```

9. cargarCursos(const string& ruta) → void

Descripción: Lee el archivo de cursos (.lfp) desde la ruta especificada y carga todos los registros en el vector global cursos.

Formato esperado: codigo,nombre,creditos,semestre,carrera

Validación: Verifica que el archivo exista; muestra mensaje de error si no.

```
// función para cargar el archivo de cursos
void cargarCursos(const string& ruta) {
    ifstream archivo(ruta);

    // verificamos si el archivo se abrió correctamente
    if (!archivo.is_open()) {
        cout << "error: no se pudo abrir el archivo de cursos" << endl;
        return;
    }

    string linea;
    int contador = 0;

    // leemos línea por línea del archivo
    while (getline(archivo, linea)) {
        // saltamos líneas vacías
        if (linea.empty()) continue;

        // separamos la línea por comas
        vector<string> datos = split(linea, ',');

        // verificamos que tengamos todos los campos
        if (datos.size() != 5) {
            cout << "advertencia: línea con formato incorrecto: " << linea << endl;
            continue;
        }

```

10. cargarNotas(const string& ruta) → void

Descripción: Lee el archivo de notas (.lfp) desde la ruta especificada y carga todos los registros en el vector global notas.

Formato esperado: carnet,codigo_curso,nota,ciclo,anio

Validación: Verifica que el archivo exista; muestra mensaje de error si no.

```cpp
// función para cargar el archivo de notas
void cargarNotas(const string& ruta) {
    ifstream archivo(ruta);

    // verificamos si el archivo se abrió correctamente
    if (!archivo.is_open()) {
        cout << "error: no se pudo abrir el archivo de notas" << endl;
        return;
    }

    string linea;
    int contador = 0;

    // leemos línea por línea del archivo
    while (getline(archivo, linea)) {
        // saltamos líneas vacías
        if (linea.empty()) continue;

        // separamos la línea por comas
        vector<string> datos = split(linea, ',');

        // verificamos que tengamos todos los campos
        if (datos.size() != 5) {
            cout << "advertencia: línea con formato incorrecto: " << linea << endl;
            continue;
        }
```

11. generarReporteEstadisticasPorCursoHTML() → void

Descripción: Genera un archivo HTML con estadísticas de cada curso: promedio, máximo, mínimo, desviación estándar y mediana.

```
// reporte 1: estadísticas generales por curso (en html)
void generarReporteEstadisticasPorCursoHTML() {
    // validación rápida para no generar html vacío
    if (cursos.empty() || notas.empty()) {
        cout << "primero carga cursos y notas para generar este reporte\n";
        return;
    }

    // ejecutado desde src
    ofstream archivo("../docs/reporte1_estadisticas_por_curso.html");

    // ejecutado desde raiz
    if (!archivo.is_open()) {
        archivo.open("docs/reporte1_estadisticas_por_curso.html");
    }

    if (!archivo.is_open()) {
        cout << "error: no se pudo crear el archivo html del reporte 1\n";
        return;
    }

    // aquí escribo la estructura básica del html
    archivo << "<!DOCTYPE html>\n";
    archivo << "<html lang=\"es\">\n";
    archivo << "<head>\n";
    archivo << "  <meta charset=\"UTF-8\">\n";
```

Archivo generado: docs/reporte1_estadisticas_por_curso.html

12. generarReporteRendimientoPorEstudianteHTML() → void

Descripción: Genera un archivo HTML mostrando el rendimiento de cada estudiante: promedio, cursos aprobados/reprobados, y créditos acumulados.

Archivo generado: docs/reporte2_rendimiento_por_estudiante.html

```
// reporte 2: rendimiento por estudiante (en html)
void generarReporteRendimientoPorEstudianteHTML() {
    // para este reporte sí necesito estudiantes, cursos y notas
    if (estudiantes.empty() || cursos.empty() || notas.empty()) {
        cout << "primero carga estudiantes, cursos y notas para generar este rep
        return;
    }

    // desde src
    ofstream archivo("../docs/reporte2_rendimiento_por_estudiante.html");

    // por si se ejecuta desde la raiz
    if (!archivo.is_open()) {
        archivo.open("docs/reporte2_rendimiento_por_estudiante.html");
    }

    if (!archivo.is_open()) {
        cout << "error: no se pudo crear el archivo html del reporte 2\n";
        return;
    }
```

13. generarReporteTop10EstudiantesHTML() → void

Descripción: Genera un archivo HTML con un ranking de los 10 mejores estudiantes ordenados por promedio descendente.

Archivo generado: docs/reporte3_top10_estudiantes.html

```cpp
// reporte 3: top 10 mejores estudiantes (en html)
void generarReporteTop10EstudiantesHTML() {
    // validaciones
    if (estudiantes.empty() || notas.empty()) {
        cout << "primero carga estudiantes y notas para generar este reporte\n";
        return;
    }

    // creo vector de estudiantes con promedios
    vector<EstudianteConPromedio> top;

    for (const Estudiante& estudiante : estudiantes) {
        vector<double> notasEstudiante;

        for (const Nota& n : notas) {
            if (n.carnet == estudiante.carnet) {
                notasEstudiante.push_back(n.nota);
            }
        }

        double promedio = calcularPromedio(notasEstudiante);
        top.push_back({estudiante, promedio});
```

14. generarReporteReprobacionHTML() → void

Descripción: Genera un archivo HTML mostrando qué cursos tienen mayor índice de reprobación, ordenados de mayor a menor porcentaje.

Archivo generado: docs/reporte4_cursos_reprobacion.html

```
// reporte 4: cursos con mayor índice de reprobación (en html)
void generarReporteReprobacionHTML() {
    if (cursos.empty() || notas.empty()) {
        cout << "primero carga cursos y notas para generar este reporte\n";
        return;
    }

    struct ReprobacionCurso {
        Curso curso;
        int totalEstudiantes;
        int aprobados;
        int reprobados;
        double porcentajeReprobacion;
    };

    vector<ReprobacionCurso> cursoReprobacion;

    for (const Curso& curso : cursos) {
        set<int> estudiantesUnicos;
        int aprobados = 0;
        int reprobados = 0;

        for (const Nota& n : notas) {
            if (n.codigo_curso == curso.codigo) {
                estudiantesUnicos.insert(n.carnet);

```

15. generarReporteAnalisisCarreraHTML() → void

Descripción: Genera un archivo HTML con un análisis agregado por carrera: total de estudiantes, promedio general, cursos disponibles y distribución por semestre.

Archivo generado: docs/reporte5_analisis_carrera.html

```cpp
// reporte 5: análisis por carrera (en html)
void generarReporteAnalisisCarreraHTML() {
    if (estudiantes.empty() || cursos.empty()) {
        cout << "primero carga estudiantes y cursos para generar este reporte\n";
        return;
    }

    set<string> carreras;
    for (const Estudiante& est : estudiantes) {
        carreras.insert(est.carrera);
    }

    ofstream archivo("../docs/reporte5_analisis_carrera.html");

    if (!archivo.is_open()) {
        archivo.open("docs/reporte5_analisis_carrera.html");
    }

    if (!archivo.is_open()) {
        cout << "error: no se pudo crear el archivo html del reporte 5\n";
        return;
    }

    archivo << "<!DOCTYPE html>\n";
    archivo << "<html lang=\"es\">\n";
    archivo << "<head>\n";
    archivo << "  <meta charset=\"UTF-8\">\n";
    archivo << "  <meta name=\"viewport\" content=\"width=device-width, initial-scale=1.0\">\n";
    archivo << "  <title>Reporte 5 - Analisis por Carrera</title>\n";
```

16. mostrarMenu() → void

Descripción: Imprime en consola el menú principal con las 9 opciones disponibles del sistema.

```cpp
// función para mostrar el menú principal
void mostrarMenu() {
    cout << "\n==========================================\n";
    cout << "   SISTEMA DE ANÁLISIS ACADÉMICO\n";
    cout << "==========================================\n";
    cout << "1. Cargar archivo de estudiantes\n";
    cout << "2. Cargar archivo de cursos\n";
    cout << "3. Cargar archivo de notas\n";
    cout << "4. Generar Reporte: Estadísticas por Curso\n";
    cout << "5. Generar Reporte: Rendimiento por Estudiante\n";
    cout << "6. Generar Reporte: Top 10 Mejores Estudiantes\n";
    cout << "7. Generar Reporte: Cursos con Mayor Reprobación\n";
    cout << "8. Generar Reporte: Análisis por Carrera\n";
    cout << "9. Salir\n";
    cout << "==========================================\n";
    cout << "Seleccione una opción: ";
}
```

17. main() → int

Descripción: Función principal que controla el flujo del programa. Implementa un bucle do-while que mantiene el menú activo hasta que el usuario seleccione opción 9 (Salir).

```
// función principal
int main() {
    cout << "Sistema de Análisis de Rendimiento Académico\n\n";

    int opcion = 0;

    // el menú se va a repetir hasta que el usuario presione 9 para salir
    do {
        mostrarMenu();
        cin >> opcion;
        cin.ignore(); // limpiamos el buffer del enter

        switch(opcion) {
            case 1:
                // cargamos el archivo de estudiantes desde la carpeta data
                cargarEstudiantes("../data/estudiantes.lfp");
                break;

            case 2:
                // cargamos el archivo de cursos desde la carpeta data
                cargarCursos("../data/cursos.lfp");
                break;

            case 3:
                // cargamos el archivo de notas desde la carpeta data
                cargarNotas("../data/notas.lfp");
                break;
```